SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 1 – CO1

Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System

CO1: Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.

Field	Details	Field	Details
Student Name	BATHULA KARTHIK	Register Number	192525286
Department / Section	B.Tech(AI&ML)	Academic Year / Semester	2026
Faculty Name	Dr.R.Latha	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date		Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Introduce Cloud Computing	3		Definition; characteristics; deployment models; scenario mapping
2	Trace Evolution and Hardware Evolution	3		Timeline; hardware evolution; relevance to university
3	Analyse Internet Software Evolution	3		Web/software evolution; SOA; APIs; scenario relevance
4	Apply Server Virtualization	5		Hypervisor types; VM/container comparison; consolidation; limitations
5	Apply Web Services	4		SOA and SOAP/REST; three university APIs; request/response examples
6	Analyse IaaS, PaaS and SaaS	5		Responsibility layers; five-application mapping; comparison; providers
7	Examine XaaS	3		Four or more variants; two justified choices; provider risks

8	Design Cloud Solution Architecture	5		Labelled diagram; deployment/service models; scaling; security; DR
9	Hands-on Implementation and Documentation	4		Running VM/container; PaaS/SaaS; tests; actual screenshots
10	Evaluate and Recommend Final Solution	7		Three-way comparison; 3-year costs; risks; phased roadmap
11	Day-4 Design Challenge	8		Impact; evaluation; adaptation; evidence-based defence
	TOTAL	**50**		**Percentage :**

EVIDENCE VERIFICATION CHECKLIST

Tasks	**Evidence expected**	**Status (✓ / P / ✗)**	**Evaluator note**
1–3	Conceptual explanations, comparisons and relevant illustrations	✓	Explanations and relevant scenario mapping documented
4	Virtualization calculations with stated assumptions and peak-load caution	✓	Calculations present; practical capacity validation recommended
5–7	API examples, service responsibility table and justified XaaS choices	✓	Required conceptual evidence documented
8	Labelled architecture diagram, caption below figure and component justification	✓	Architecture documented; sizing assumptions need testing
9	Actual code, environment details, execution logs, API outputs and screenshots	P	Procedures provided; actual execution evidence not established
10	Cost assumptions, 3-year totals, risk controls and migration roadmap	✓	Estimates and rationale documented
11	Day-4 changes, 10× load strategy, 99.9% target and 40-college isolation	✓	Design analysis documented; practical validation pending

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	**Observation**
Source code	✓	Flask REST API and Dockerfile provided in the proposed implementation
Virtualization	P	Docker container configuration documented; running-container evidence required
PaaS or SaaS	P	Managed web-service deployment procedure described; live

		deployment not verified
Functional test	P	Expected HTTP 200 response and JSON output documented; actual output required
Failure case	P	HTTP 404 test case documented; actual execution record required
Screenshots	P	Screenshot requirements identified; authentic screenshots need verification
Reproducibility	✓	Build commands, run commands, endpoints and expected results specified

OVERALL REMARKS

K.Sasidhar Reddy **Student Signature**	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Cloud definition, characteristics, deployment models and scenario mapping	✓	All conceptual requirements covered
2	Evolution timeline, hardware development and university relevance	✓	Well explained with relevant examples
3	Internet software evolution, SOA and APIs	✓	Clear explanation and scenario relevance
4	Hypervisors, VM/container comparison and consolidation calculations	✓	Calculations included; peak-load limitations discussed
5	SOAP/REST comparison and three university APIs	✓	API methods and request/response examples documented
6	IaaS/PaaS/SaaS responsibility layers and application mapping	✓	All five applications categorized with justification
7	XaaS variants and two justified selections	✓	Six variants explained; DBaaS and DRaaS recommended
8	Labelled architecture, scaling, security and disaster recovery	✓	Design and component justifications documented
9	Working container, PaaS/SaaS deployment, actual tests and screenshots	P	Code and procedures provided; actual execution evidence requires verification
10	Three-year cost comparison, risks and migration roadmap	✓	Cost assumptions, calculations and recommendations included
11	Impact analysis, 10× load, 99.9% availability and tenant isolation	✓	Changed requirements evaluated; load-test evidence recommended

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Source code	✓	Python Flask REST API and Dockerfile documented
Virtualization	P	Docker build and run procedures provided; execution screenshot required
PaaS or SaaS	P	Cloud deployment steps described; actual deployment confirmation required

Functional test	P	HTTP 200 and expected JSON response documented; actual test output required
Failure case	P	HTTP 404 test case included; execution evidence required
Screenshots	P	Required screenshots identified but authenticity and completeness not verified
Reproducibility	✓	Installation, configuration, commands and testing steps documented
Interpretation	✓	Technical outcomes and implementation limitations explained
Code and report consistency	✓	Proposed API, Docker configuration and testing procedure are aligned

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	**Status**
All 11 tasks answered	☒ Yes ☐ Partly ☐ No
Cloud concepts and comparisons explained	☒ Yes ☐ Partly ☐ No
Virtualization calculations included	☒ Yes ☐ Partly ☐ No
API examples and service models explained	☒ Yes ☐ Partly ☐ No
Cloud architecture diagram included	☒ Yes ☐ Partly ☐ No
Implementation and testing evidence provided	☐ Yes ☒ Partly ☐ No
Three-year cost analysis and roadmap included	☒ Yes ☐ Partly ☐ No
Day-4 design challenge completed	☒ Yes ☐ Partly ☐ No
Figures and screenshots included	☐ Yes ☒ Partly ☐ No
Table and figure numbering correct	☐ Yes ☒ Partly ☐ No
Viva recommended	☒ Yes ☐ No

CSA15 – Cloud Computing and Big Data Analytics

ASSIGNMENT 1

CO1: *Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.*

Assignment Title: Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System (Total Marks – 50)

Duration: 1 week

Scenario – Smart Campus Cloud Migration

A multi-campus university currently runs its Student Information System, Learning Management System, Library System, Exam Portal and Email on ageing physical servers in a single on-premise data centre. Each application has its own dedicated server, which stays idle for most of the year but fails to cope during admissions, result publication and online-exam weeks.

The university's current environment is:

- 12 physical servers, average CPU utilisation of about 15%, with peaks above 95% during admissions and exams
- Approximately 25,000 students and 2,000 staff across 4 campuses
- Applications built as large monolithic programs, with no public interfaces for mobile apps or partner colleges
- High capital expenditure, slow procurement (6–8 weeks per server) and no disaster recovery site

As a cloud consultant, you must study how computing evolved to the cloud model, and then design a cloud-based solution that uses virtualization, web services and an appropriate mix of service models (IaaS, PaaS, SaaS and XaaS) for the university. All recommendations must be justified in terms of cost, scalability, availability and manageability.

Design Challenge:

On Day 4, the university announces that it will launch a mobile app and open student-record and fee-payment services to 40 affiliated colleges. Expected load is 10 times the current peak, with a requirement of 99.9% availability and a limited operating budget. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Introduce Cloud Computing and Map It to the Scenario (3 Marks)

Design and Propose cloud computing model for the context of the university's problems.

- Definition of cloud computing and the five essential characteristics (on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service)
- Deployment models (public, private, hybrid, community) and which one suits the university
- Three problems of the current on-premise setup and how each cloud characteristic addresses them
- Benefits and limitations of cloud adoption for the university

Evidence Expected

A clear explanation of cloud characteristics and deployment models, mapped to the university's actual problems.

TASK 2 – Trace the Evolution and Hardware Evolution (3 Marks)

Trace how computing infrastructure evolved to make cloud computing possible.

- A timeline from mainframes, minicomputers and PCs, to client–server, clusters, grid computing, utility computing and cloud
- Hardware evolution: multi-core processors, commodity servers, high-capacity storage, high-speed networking and data centres
- How each stage solved a limitation of the previous one
- Which stage the university's current system corresponds to, with reasons

Evidence Expected

A labelled timeline or diagram, with a justified explanation of how hardware advances enabled the cloud.

TASK 3 – Analyse Internet Software Evolution (3 Marks)

Explain how internet software evolved and why it supports cloud services.

- Evolution from static web pages to dynamic web, Web 2.0, SOA, mashups and cloud-hosted applications
- Role of browsers, HTTP, AJAX and APIs in delivering software over the internet
- Comparison between desktop software, client–server software and web-delivered software
- How this evolution affects the design of the university's applications

Evidence Expected

A comparative explanation of software evolution stages, linked to the university's application design.

TASK 4 – Apply Server Virtualization (5 Marks)

Apply server virtualization to consolidate the university's 12 physical servers.

- Explain how a hypervisor works, and distinguish Type-1 and Type-2 hypervisors, with examples
- Compare full virtualization, para-virtualization, OS-level virtualization (containers) and hardware-assisted virtualization
- Propose which applications should run as virtual machines and which as containers, with reasons
- Calculate the consolidation: with an average utilisation of 15% per server and a target host utilisation of 70%, estimate how many physical hosts are needed (show your assumptions and working)
- Discuss live migration, snapshots and resource over-commitment as benefits, and the risks of virtualization (noisy neighbour, hypervisor failure, security isolation)

Show your calculation. A final number without working will not receive full credit.

Evidence Expected

A justified virtualization plan with a worked consolidation calculation and discussion of benefits and risks.

TASK 5 – Apply Web Services (4 Marks)

Analyse web services to expose the university's functions to other systems.

- Overview of web services and their role in cloud and SOA
- Compare SOAP, WSDL, UDDI and RESTful services (HTTP methods, JSON/XML)
- Identify at least three university functions to expose as web services (for example, result lookup, fee payment, library search) and define the endpoint, method and sample request/response for each
- Justify whether SOAP or REST is more suitable for each function

Evidence Expected

A defined set of web services with sample request/response formats and a justified choice of SOAP or REST

TASK 6 – Analyse the Service Models: IaaS, PaaS and SaaS (5 Marks)

Analyse which service model fits each university application.

- Define IaaS, PaaS and SaaS, and show who manages which layer (hardware to application) in each model

- Categorise the five university applications (SIS, LMS, Library, Exam Portal, Email) into IaaS, PaaS or SaaS, with reasons
- Compare the models on control, cost, scalability, maintenance effort and vendor lock-in
- Give one real provider example for each model

Present your **categorisation in a table**, but explain the **reasoning behind** each **decision**. A table alone is not sufficient.

Evidence Expected

A reasoned mapping of applications to service models, supported by a layer-wise responsibility comparison.

TASK 7 – Examine XaaS (3 Marks)

Examine the broader XaaS (Anything-as-a-Service) concept for the university.

- Explain XaaS and describe at least four variants (for example DaaS, BaaS/MBaaS, FaaS, DBaaS, Security-as-a-Service, Disaster-Recovery-as-a-Service)
- Select two XaaS offerings that would benefit the university, and justify them
- Discuss the risks of over-reliance on multiple 'as-a-Service' providers

Evidence Expected

An analysis of XaaS variants, with justified selection of two offerings for the university.

TASK 8 – Design the Cloud Solution Architecture (5 Marks)

Bloom's Level: Level 6 – Create | *Action verbs: Design, Construct, Propose*

Create a complete cloud architecture for the university by combining virtualization, web services and service models.

- An architecture diagram showing users, load balancer, web/application tier, database tier, storage, API layer and DR region
- The deployment model and the service model chosen for each component
- How elasticity (auto-scaling) will handle admissions and exam peaks
- Basic security and availability measures (access control, encryption, backup, multi-zone deployment)

Evidence Expected

A labelled architecture diagram with written justification for each component and design decision.

TASK 9 – Hands-on Implementation and Documentation (4 Marks)

Implement a small proof-of-concept of your design using a free-tier cloud account or a local virtualization tool.

- Create a virtual machine or container (using AWS/Azure/GCP free tier, VirtualBox, or Docker) and host a simple web page or REST API for one university function
- Deploy a SaaS or PaaS component (for example a cloud-hosted database, a serverless function or a static site)
- Test the web service with a client (browser, Postman or curl) and record the output
- Provide screenshots, configuration steps and any issues faced

Evidence Expected

Annotated screenshots and clear, reproducible steps showing that the proof-of-concept works.

TASK 10 – Evaluate and Recommend the Final Solution (7 Marks)

Evaluate your design and recommend the final cloud solution for the university.

- Compare on-premise, IaaS-based and PaaS/SaaS-based options on cost (CapEx vs OpEx), scalability, availability, security and effort
- Estimate a simple 3-year cost comparison using stated assumptions
- Identify the main risks (vendor lock-in, data privacy, downtime, skills gap) and give mitigation steps
- Provide your final recommendation and a short migration roadmap (which application moves first, and why)

Your recommendation must be supported by the analysis in the previous tasks. Choosing a popular cloud provider without justification is not sufficient.

Evidence Expected

A technically justified recommendation, supported by a cost comparison, risk analysis and migration roadmap.

DAY-4 DESIGN CHALLENGE

TASK 11 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

The university will now launch a mobile app and open student-record and fee-payment services to 40 affiliated colleges. Expected load is 10 times the current peak, availability must be 99.9%, and the operating budget is limited.

Re-evaluate your original design in response to these changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions (load, users, availability, cost) are affected
- Why the web services and APIs now become critical
- How multi-tenant access by 40 colleges affects security and isolation

Evidence Expected: A clear analysis of how the changed requirements affect the original cloud design.

Evaluation of the Original Design – 2 Marks

Discuss:

- Whether the chosen service models and virtualization plan can handle a 10x load
- Whether auto-scaling and the single-region deployment are sufficient for 99.9% availability (about 8.76 hours of downtime allowed per year)
- Whether cost remains acceptable under the new load
- Whether another service model or XaaS offering would give better overall results

Evidence Expected: A technically supported evaluation of the original design under the new conditions.

Adaptation or Redesign – 2 Marks

Discuss:

- Decide whether to retain, modify or redesign the solution
- Describe the revised architecture (for example API gateway, container orchestration, serverless components, multi-zone deployment or a hybrid model)
- Explain what changed in the service model mix and in the virtualization approach
- If you retain the original design, give sufficient technical justification for why no change is required

Evidence Expected: An appropriate design response to the changed requirement, with clear technical reasoning.

Evidence-Based Defence – 2 Marks

Discuss:

- Capacity and availability calculations

- Cost comparison between the original and the revised design
- Security and multi-tenancy considerations
- Trade-offs such as vendor lock-in, complexity and migration effort

Evidence Expected: A well-supported final decision that follows logically from the analysis.

MARK DISTRIBUTION

Task	Title	Bloom's Level	Marks
Task 1	Cloud Computing – Introduction & Characteristics	L2 – Understand	3
Task 2	Evolution & Hardware Evolution	L2 – Understand	3
Task 3	Internet Software Evolution	L2 – Understand	3
Task 4	Server Virtualization	L3 – Apply	5
Task 5	Web Services	L3 – Apply	4
Task 6	Service Models: IaaS, PaaS, SaaS	L4 – Analyze	5
Task 7	XaaS	L4 – Analyze	3
Task 8	Design the Cloud Solution Architecture	L6 – Create	5
Task 9	Hands-on Implementation & Documentation	L3 – Apply	4
Task 10	Evaluate & Recommend the Final Solution	L5 – Evaluate	7
	Subtotal		

Task 11	Day-4 Design Challenge	L4 / L5 / L6	8
	TOTAL		50

TASK 1 – Introduce Cloud Computing and Map It to the Scenario

1.1 Definition and the five essential characteristics

Cloud computing is a model for delivering computing resources (servers, storage, databases, networking, software and analytics) over the internet, on a pay-as-you-use basis, from a shared pool that can be provisioned and released rapidly with minimal management effort. For the university, this means that instead of buying and housing 12 physical servers, the IT team rents exactly the capacity it needs, at the moment it needs it.

Table 1: Five essential characteristics of cloud computing mapped to the university

Characteristic	Meaning	How it helps the university
On-demand self-service	IT staff provision servers, storage or databases through a console or API without contacting the provider.	A new server is ready in minutes instead of the present 6–8 week procurement cycle.
Broad network access	Services are available over the network from phones, laptops and tablets using standard protocols.	Students, staff and partner colleges on all 4 campuses reach the same services; the mobile app becomes possible.
Resource pooling	The provider serves many customers from a shared pool of hardware (multi-tenancy) and assigns resources dynamically.	Idle capacity is no longer locked inside one application; replaces 12 servers that average only about 15% load.
Rapid elasticity	Capacity is added or removed automatically as demand rises and falls.	Admission, result and exam weeks (CPU above 95%) are absorbed by scaling out, then scaled back in.
Measured service	Usage is metered and billed per use; usage reports can be produced per department.	Turns large capital expenditure into operating expenditure and enables campus-wise cost tracking.

1.2 Deployment models and the model that suits the university

Table 2: Cloud deployment models and their suitability

Model	Description	Suitability for the university
Public	Resources owned and operated by a third-party provider and shared by many customers over the internet.	Highly elastic and low cost; suits the web tier, exam-week bursts and SaaS applications. Data-sensitivity and compliance must be managed.
Private	Infrastructure used exclusively by one organisation, on-premise or hosted.	Maximum control, but high capital cost and limited elasticity. The current data centre is effectively a private, non-elastic set-up.
Hybrid	A private environment and a public cloud connected so that workloads and data can move between them.	**Best fit.** Sensitive identity services and legacy devices stay private; elastic and externally facing workloads run in the public cloud.
Community	Infrastructure shared by several organisations with common needs (for example, education institutions).	Useful later for the 40 affiliated colleges sharing one academic platform; not the primary model.

Recommended model: Hybrid cloud. Public cloud (an India-based region) is used for elastic and externally facing services, while a small consolidated virtualised cluster (sized in Task 4) is retained for identity services and legacy integrations during migration. This choice keeps student data under tight control, supports India's data-protection obligations (Digital Personal Data Protection Act, 2023), allows a low-risk phased migration, and still removes the cost of owning peak-sized hardware.

1.3 Three problems of the on-premise set-up and the cloud characteristic that addresses each

Table 3: Problems of the current environment and their cloud solutions

Problem in the current set-up	Cloud characteristic	How it is addressed
Very low utilisation (about 15% average) and high capital cost, because every application owns a dedicated server.	Resource pooling and measured service	Applications share a pooled, virtualised resource base and the university pays only for what is consumed, so no idle hardware is owned.
Servers fail under peak load (above 95% CPU) during admissions, result publication and online exams.	Rapid elasticity	Auto-scaling adds instances before and during peaks and removes them afterwards.
6–8 week procurement per server and no disaster-recovery site.	On-demand self-service and broad network access	Environments are created in minutes; data can be replicated to a second region for recovery and reached from any campus or device.

1.4 Benefits and limitations of cloud adoption

Benefits

- Lower upfront cost and pay-per-use billing instead of buying peak-sized hardware.
- Elastic capacity that matches admissions, result and exam peaks automatically.
- Faster deployment of new services (minutes rather than weeks).
- Built-in backup, multi-zone availability and disaster recovery that the university does not have today.
- Anywhere access, which enables the mobile app and integration with partner colleges.
- Managed services (database, security, monitoring) that reduce routine administration.

Limitations

- Dependence on reliable internet connectivity at all campuses.
- Vendor lock-in if proprietary services are used heavily.
- Data privacy, compliance and data-residency obligations for student records.
- Costs can grow unexpectedly if usage is not monitored and capped.
- Skills gap: staff need training in cloud operations and security.

- Shared-responsibility security model and less control over the underlying infrastructure.

TASK 2 – Trace the Evolution and Hardware Evolution

2.1 Timeline of computing infrastructure

Figure 1 shows how computing infrastructure moved from centralised mainframes to the cloud. Each stage removed a limitation of the one before it.

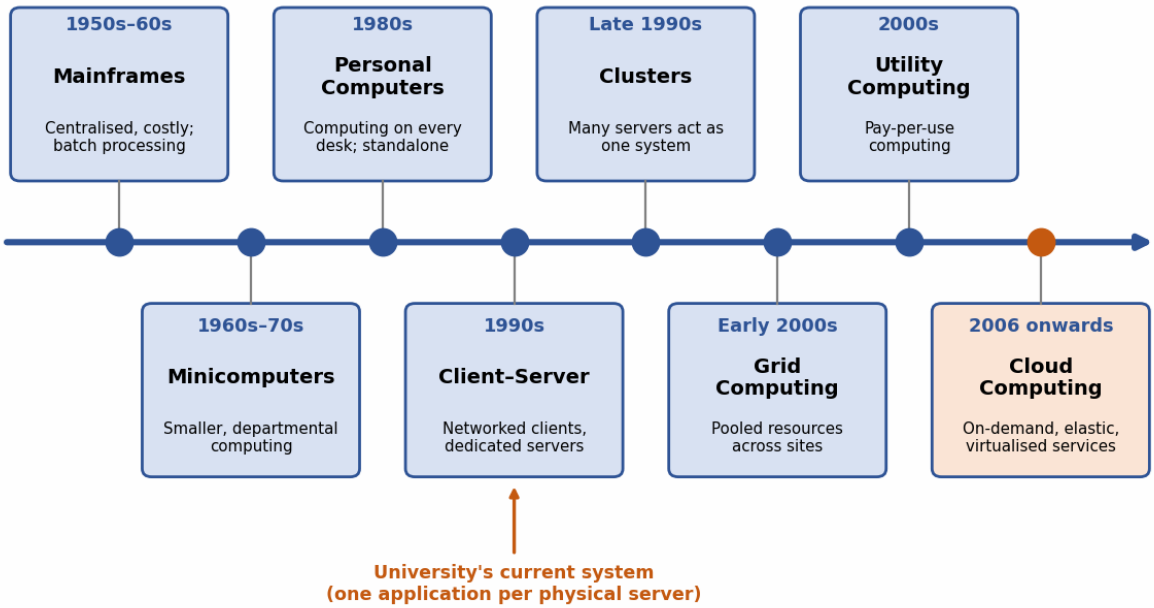


Figure 1: Evolution of computing infrastructure from mainframes to cloud computing

Table 4: How each stage solved a limitation of the previous stage

Stage	Period	Key idea	Limitation of previous stage it solved	New limitation
Mainframes	1950s–60s	Large central computer shared through terminals; batch processing.	Manual, one-job-at-a-time calculation.	Very expensive; central bottleneck.
Minicomputers	1960s–70s	Smaller, cheaper machines for departments.	Mainframe cost and central dependence.	Isolated departmental islands.
Personal computers	1980s	A computer on every desk with graphical software.	Dependence on shared central machines.	Data not shared; hard to administer.
Client–server	1990s	Clients on a LAN request services from dedicated servers.	Isolated PCs and unshared data.	One server per application; poor scaling.
Clusters	Late 1990s	Many commodity servers working as one system.	Capacity and fault-tolerance limits of one server.	Fixed size; tied to one site and owner.

Grid computing	Early 2000s	Resources of several organisations coordinated for large jobs.	A cluster is limited to a single site.	Complex; mostly batch and scientific use.
Utility computing	2000s	Computing metered and billed like electricity.	Fixed up-front purchase of capacity.	Little self-service; virtualisation immature.
Cloud computing	2006 onwards	On-demand, elastic, metered services delivered over the internet using virtualisation.	Combines clusters, grid and utility ideas with self-service and elasticity.	Needs internet, governance and cost control.

2.2 Hardware evolution that made the cloud possible

Table 5: Hardware advances and their relevance to cloud computing

Hardware advance	What changed	Why it matters for the cloud
Multi-core processors	From the mid-2000s, CPUs carry many cores and include hardware support for virtualisation (for example Intel VT-x, AMD-V).	Many virtual machines can run on one host at close to native speed.
Commodity servers	Standard low-cost x86 servers replaced proprietary hardware.	Providers scale out cheaply and handle failures in software rather than expensive hardware.
High-capacity storage	Larger disks, SSDs, SAN and distributed object storage.	Cheap, durable storage for data, backups and virtual-machine images.
High-speed networking	From gigabit to 10/40/100 Gbps Ethernet; widespread broadband.	Makes remote data centres practical and keeps latency low for users.
Data centres	Warehouse-scale facilities with automated power, cooling and management.	Economies of scale and multi-site redundancy that no single university can match.

2.3 Stage corresponding to the university's current system

The current environment corresponds to the **client–server stage (1990s style)**, with no cloud features. The reasons are:

- Each application (SIS, LMS, Library, Exam Portal, Email) runs on its own dedicated physical server.
- All servers sit in a single on-premise data centre, so resources are never shared (about 15% average utilisation).
- Capacity is fixed: it cannot scale out for peaks above 95% or shrink afterwards.
- No virtualisation, no self-service (6–8 weeks to add a server) and no disaster-recovery site.

- Applications are monolithic programs without public interfaces, typical of client–server design.

TASK 3 – Analyse Internet Software Evolution

3.1 Evolution of internet software

Table 6: Stages in the evolution of internet software

Stage	Period	Main characteristics	Contribution to cloud services
Static web pages (Web 1.0)	1990s	Read-only HTML pages served by web servers.	Universal access through a browser and HTTP.
Dynamic web	Late 1990s	Server-side scripting (CGI, PHP, JSP, ASP) with databases generates pages on demand.	Application logic moves to servers, creating true web applications.
Web 2.0	Mid-2000s	Interactive, user-generated content; AJAX gives desktop-like interfaces.	Rich web applications make software-as-a-service practical.
SOA and web services	2000s	Applications built from loosely coupled services with standard interfaces (SOAP/WSDL, later REST).	Reusable, interoperable services, the basis of cloud APIs.
Mashups	2000s	New applications combine data and services from several APIs (for example maps plus events).	Shows APIs as building blocks that can be reused.
Cloud-hosted applications	2006 onwards	Applications delivered as services on elastic, pay-per-use infrastructure (SaaS, PaaS).	Scalable, always-on software without local installation.

3.2 Role of browsers, HTTP, AJAX and APIs

- **Browsers** act as a universal client, so users need no installation and software runs on any device.
- **HTTP** is the standard request–response protocol; its methods (GET, POST, PUT, DELETE) and status codes (200, 404, 500) also underpin REST APIs.
- **AJAX** lets a page exchange small amounts of data with the server in the background, giving responsive interfaces without full page reloads.
- **APIs** (REST/JSON or SOAP/XML) expose application functions to mobile apps, partner systems and other services in a machine-readable way.

3.3 Desktop, client–server and web-delivered software

Table 7: Comparison of desktop, client–server and web-delivered software

Aspect	Desktop software	Client–server software	Web-delivered (cloud) software
Installation	On every PC.	Client on every PC; server in the data centre.	None; browser or mobile app.

Updates	Manual, machine by machine.	Client and server updated separately.	Central; all users updated at once.
Platform dependence	Tied to one operating system.	Often tied to the client operating system.	Independent of device and operating system.
Scalability	Limited to one machine.	Limited by the dedicated server.	Elastic; scales with demand.
Access	Local machine only.	Within the campus network.	Anywhere with internet access.
Cost model	Licence per PC.	Licence plus server capital cost.	Subscription or pay-per-use.
University example	Marks entry in a spreadsheet on a staff PC.	Present SIS with thick client and dedicated server.	SaaS LMS and email; mobile app.

3.4 Effect on the design of the university's applications

- Break the monolithic programs into modular, loosely coupled services with documented, versioned APIs.
- Make the application tier **stateless** (sessions kept in a shared cache) so that instances can be added or removed freely.
- Provide browser-based and responsive or mobile front ends instead of thick clients.
- Use single sign-on (SAML or OpenID Connect) across SIS, LMS, Library and Email.
- Adopt automated build and deployment (CI/CD) so that updates reach all campuses at once.

TASK 4 – Apply Server Virtualization

4.1 How a hypervisor works

A **hypervisor** (virtual machine monitor) is a software layer that sits between the physical hardware and the guest operating systems. It divides the host's CPU, memory, storage and network among several isolated **virtual machines (VMs)**, schedules their access to the hardware, and keeps each VM unaware of the others. Each VM runs its own operating system and applications as if it had a dedicated server.

Table 8: Type-1 and Type-2 hypervisors

Feature	Type-1 (bare-metal)	Type-2 (hosted)
Where it runs	Directly on the physical hardware.	As an application on top of a host operating system.
Performance	High, with little overhead.	Lower, because of the extra host-OS layer.
Typical use	Production data centres and cloud platforms.	Desktops, development, testing and learning.
Examples	VMware ESXi, Microsoft Hyper-V, KVM, Xen.	Oracle VirtualBox, VMware Workstation.

Suitability here	**Used for the university's consolidated hosts.**	Suitable only for staff testing and the proof-of-concept.

4.2 Virtualization techniques compared

Table 9: Full, para-, OS-level and hardware-assisted virtualization

Technique	How it works	Strengths	Weaknesses / examples
Full virtualization	The hypervisor emulates the hardware fully; the guest OS runs unmodified.	Any guest OS works; strong isolation.	Some overhead if done without hardware help. VMware ESXi, KVM.
Para-virtualization	The guest OS is modified to call the hypervisor directly (hypercalls).	Lower overhead; efficient I/O.	Needs a modified guest OS. Xen (PV mode).
OS-level virtualization (containers)	Containers share the host kernel but have isolated processes, file systems and networks.	Very light; start in seconds; high density; ideal for scaling.	Weaker isolation (shared kernel); same OS family. Docker, LXC.
Hardware-assisted virtualization	CPU extensions (Intel VT-x, AMD-V) let guests run directly on the processor.	Near-native speed; simplifies the hypervisor; enables full virtualization without emulation.	Needs a supporting CPU and BIOS setting. It is an enabling feature rather than a separate product.

4.3 Which applications run as VMs and which as containers

Table 10: Proposed virtualization plan for the five applications

Workload	Choice	Reason
Student Information System – database	VM	Stateful, critical, needs strong isolation, dedicated memory and predictable I/O.
Student Information System – web/app tier	Containers	Can be made stateless; scales out quickly during admissions.
Online Exam Portal	Containers (initially VMs for the proctoring components)	Highly bursty, so fast start-up and horizontal scaling are essential; components needing OS-level agents stay on VMs.
Learning Management System	VM during migration, then SaaS	Legacy monolith with plug-ins runs best on a VM until it moves to a hosted LMS.
Library System	Container	Light workload and low traffic; small footprint.
Email	VM during migration, then SaaS	Mail servers are long-running, stateful services; the target is a hosted email service.

4.4 Consolidation calculation

Assumptions

- All 12 existing servers have equal capacity, so **one server = 1 server-unit (SU)**.

- Each new virtualisation host is sized equal to one existing server (1 SU). This is conservative, because a modern host is normally more powerful.
- Average utilisation per server = 15%; target host utilisation = 70% (given).
- Virtualisation overhead is taken as 10% for a sensitivity check (an assumption).
- Highly available operation needs one spare host (N+1) so that a host can fail or be taken down for maintenance.

Working

- **Step 1 – Total average workload** = 12 servers × 15% = 12 × 0.15 = **1.8 SU**.
- **Step 2 – Hosts needed at 70% target utilisation** = 1.8 ÷ 0.70 = 2.57, which rounds up to **3 hosts**.
- **Step 3 – Sensitivity with 10% overhead:** 1.8 × 1.10 = 1.98 SU; 1.98 ÷ 0.70 = 2.83, which still rounds up to **3 hosts**.
- **Step 4 – Add N+1 redundancy:** 3 + 1 = **4 hosts**.
- **Step 5 – Check:** normal utilisation = 1.8 ÷ 4 = 45%; with one host down = 1.8 ÷ 3 = 60%, which is below the 70% target, so the plan is safe.
- **Step 6 – Reduction:** 12 → 4 hosts, i.e. (12 − 4) ÷ 12 = **66.7% fewer physical servers**.

Peak-load caution. The calculation above uses average load. During admissions and exams some servers exceed 95%. If, for example, 6 of the 12 workloads peak at 95% while the other 6 stay at 15%, the load is 6 × 0.95 + 6 × 0.15 = 6.6 SU, and 6.6 ÷ 0.70 = 9.4, so about **10 hosts** would be needed. Four hosts therefore cannot absorb peaks on their own. Buying 10 hosts for a few weeks a year is wasteful, so peaks should be handled by auto-scaling in the public cloud (hybrid model). Real capacity must be validated by load testing.

Table 11: Summary of the consolidation calculation

Scenario	Load (SU)	Hosts at 70% target	Remark
Average load	1.8	2.57 → 3	Baseline
Average load + 10% overhead	1.98	2.83 → 3	Sensitivity check
Average load with N+1 spare	1.8	3 + 1 = 4	Recommended on-premise/private cluster
Peak (6 workloads at 95%)	6.6	9.43 → 10	Exceeds 4 hosts, so burst to the public cloud

4.5 Benefits and risks of virtualization

Benefits

- **Live migration:** a running VM moves to another host without downtime, so hosts can be patched or repaired without interrupting users.

- **Snapshots:** a point-in-time copy of a VM allows quick rollback after a failed upgrade (a snapshot is not a replacement for backup).
- **Resource over-commitment:** more virtual CPUs or memory can be allocated than physically exist (for example 3:1 for CPU), because workloads are rarely busy at the same time. This raises utilisation.

Risks and controls

Table 12: Risks of virtualization and their controls

Risk	Explanation	Control
Noisy neighbour	One VM consumes excessive CPU, memory or I/O and slows others on the same host.	Resource limits and reservations, monitoring, separating exam workloads from other workloads.
Hypervisor or host failure	All VMs on that host fail together.	Clustered hosts with high availability, N+1 spare capacity and live migration.
Security isolation	A hypervisor flaw (VM escape) or a shared container kernel can expose other workloads.	Prompt patching, hardened hypervisor, network segmentation, VMs for sensitive databases, least-privilege containers.
Over-commitment	Memory over-commit can cause swapping and slowdowns at peak.	Conservative ratios for memory, alerts at 70% utilisation.

TASK 5 – Apply Web Services

5.1 Web services in cloud and SOA

A **web service** is a software function made available over a network through standard protocols so that different applications, written in different languages and running on different platforms, can use it. In **service-oriented architecture (SOA)** an application is built from such loosely coupled services. In the cloud, web services and APIs are the way every service is consumed, so exposing the university's functions as web services lets the mobile app and the 40 affiliated colleges use them without access to the underlying systems.

5.2 SOAP, WSDL, UDDI and REST compared

Table 13: SOAP, WSDL, UDDI and RESTful services

Technology	What it is	Data format	Notes
SOAP	An XML messaging protocol with a strict envelope, normally carried over HTTP.	XML	Heavier messages; built-in standards for security (WS-Security) and reliable messaging; formal contracts.
WSDL	An XML document that describes a SOAP service: its operations, messages and endpoints.	XML	Machine-readable contract; tools can generate client code.
UDDI	A registry where services are published and discovered.	XML	Rarely used today; replaced by API catalogues and developer portals.

REST	An architectural style: resources are identified by URLs and handled with HTTP methods (GET, POST, PUT, PATCH, DELETE).	JSON (most common) or XML	Lightweight, stateless, cacheable; ideal for mobile and web clients.

5.3 University functions exposed as web services

Table 14: Three university APIs

No.	Function	Method and endpoint	Purpose
1	Result lookup	GET /api/v1/students/{regNo}/results?semester=5	Returns a student's results for a semester.
2	Fee payment	POST /api/v1/fees/payments	Creates a fee payment and returns a receipt reference.
3	Library search	GET /api/v1/library/books?title=cloud&limit=2	Searches the catalogue and shows availability.

API 1 – Result lookup (sample request and responses)

```
GET /api/v1/students/192370001/results?semester=5 HTTP/1.1
Host: api.university.example
Authorization: Bearer <access_token>
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json
{
  "registerNumber": "192370001",
  "semester": 5,
  "gpa": 8.6,
  "subjects": [
    {"code": "CSA15", "grade": "A"},
    {"code": "CSA11", "grade": "A+"}
  ]
}

HTTP/1.1 404 Not Found
{"error": "Student not found"}
```

API 2 – Fee payment (sample request and response)

```
POST /api/v1/fees/payments HTTP/1.1
Host: api.university.example
Authorization: Bearer <access_token>
Idempotency-Key: 7f3c-a91b-42d0
Content-Type: application/json
{
  "registerNumber": "192370001",
  "feeType": "TUITION",
  "term": "2026-ODD",
  "amount": 45000.00,
  "currency": "INR",
  "paymentMethod": "UPI"
}

HTTP/1.1 201 Created
{
  "paymentId": "PAY-20261030-000123",
  "status": "SUCCESS",
  "amount": 45000.00,
  "receiptUrl": "/api/v1/fees/receipts/PAY-20261030-000123"
```

```
}
```

API 3 – Library search (sample request and response)

```
GET /api/v1/library/books?title=cloud&limit=2 HTTP/1.1
Host: api.university.example
Accept: application/json

HTTP/1.1 200 OK
{
  "total": 14,
  "items": [
    {"isbn": "9780000000001", "title": "Cloud Computing Fundamentals", "available": true},
    {"isbn": "9780000000002", "title": "Big Data Analytics", "available": false}
  ]
}
```

All data above is illustrative sample data.

5.4 SOAP or REST for each function

Table 15: Choice of SOAP or REST for each function

Function	Choice	Justification
Result lookup	REST	Read-only GET request; JSON is compact for the mobile app; responses can be cached during result publication to reduce load.
Fee payment	REST for the app and colleges; SOAP only where a bank or college ERP requires it	REST over HTTPS with OAuth 2.0, tokens and an idempotency key is simple for mobile clients and prevents duplicate charges. SOAP with WS-Security is justified only for the back-end bank settlement interface where a formal WSDL contract and message-level security are mandated.
Library search	REST	Simple, public, read-only query with no need for message-level security; easy to consume from any partner library or app.

For the bank-facing back-end integration, a SOAP message would look like this:

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:pay="http://university.example/payment">
  <soapenv:Header><!-- WS-Security header --></soapenv:Header>
  <soapenv:Body>
    <pay:SettlePaymentRequest>
      <pay:paymentId>PAY-20261030-000123</pay:paymentId>
      <pay:amount>45000.00</pay:amount>
    </pay:SettlePaymentRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

TASK 6 – Analyse the Service Models: IaaS, PaaS and SaaS

6.1 Definitions and responsibility layers

- **IaaS (Infrastructure as a Service):** the provider supplies virtualised servers, storage and networking; the customer manages the operating system, middleware, runtime, data and applications.

- **PaaS (Platform as a Service):** the provider also manages the operating system, middleware and runtime; the customer deploys and manages only the application and its data.
- **SaaS (Software as a Service):** the provider delivers a complete application over the internet; the customer only configures it and manages its own content and users.

Table 16: Who manages which layer

Layer	On-premise	IaaS	PaaS	SaaS
Application	You	You	You	Provider
Data	You	You	You	You (content and settings)
Runtime	You	You	Provider	Provider
Middleware	You	You	Provider	Provider
Operating system	You	You	Provider	Provider
Virtualization	You	Provider	Provider	Provider
Servers	You	Provider	Provider	Provider
Storage	You	Provider	Provider	Provider
Networking	You	Provider	Provider	Provider

6.2 Mapping the five applications to service models

Table 17: Service-model categorisation of the university applications

Application	Service model	Main reason
Student Information System (SIS)	PaaS	Custom university logic, so it cannot simply be bought, but the team gains nothing from managing operating systems.
Learning Management System (LMS)	SaaS	A standard, mature product category; the provider handles scaling and upgrades.
Library System	SaaS	Standard cataloguing and circulation with low load and little customisation.
Exam Portal	IaaS	Needs OS-level control for secure-browser and proctoring components, and pre-scaled capacity before exams.
Email	SaaS	A commodity service whose security, spam filtering and availability are best left to a specialist.

Reasoning behind each decision

- **SIS → PaaS.** The SIS holds the university's own admission, enrolment, fee and result rules, so a generic package would not fit. A managed application platform with a managed database (DBaaS) lets the team deploy refactored code and scale it automatically while the provider patches operating systems and runtimes. The university keeps control of code and data. The main risk is platform lock-in, which is reduced by using standard languages and containers.

- **LMS → SaaS.** Course delivery, quizzes and assignments are standard functions available as mature hosted products. SaaS removes the need to run servers, scales for term-start peaks without effort and includes mobile apps. Integration with the SIS is done through APIs and single sign-on.
- **Library → SaaS.** Catalogue search and book circulation are routine functions with low and steady load. A dedicated server for such a light workload is wasteful, whereas a hosted library service gives updates, backup and availability at a small subscription.
- **Exam Portal → IaaS.** Exam integrity is critical. The portal needs control over the operating system (lock-down, proctoring agents, network rules) and must be scaled up before an exam starts rather than reactively. Virtual machines in an auto-scaling group with scheduled scaling give that control, and the portal can be containerised later.
- **Email → SaaS.** Running mail servers brings constant security, spam and deliverability work. A hosted education email service provides a high-availability SLA, filtering and large mailboxes and frees staff for higher-value work.

6.3 Comparison of the models

Table 18: IaaS, PaaS and SaaS compared

Criterion	IaaS	PaaS	SaaS
Control	High (from the operating system upwards).	Medium (application and data).	Low (configuration only).
Cost	Pay for allocated VMs even when lightly used; higher administration cost.	Pay for usage of the platform; lower administration cost.	Per-user subscription; lowest administration cost.
Scalability	Must be designed (auto-scaling groups, load balancers).	Largely automatic within platform limits.	Handled entirely by the provider.
Maintenance effort	High: patching, hardening, monitoring.	Medium: application and data only.	Minimal.
Vendor lock-in	Low to medium; VM images are fairly portable.	Medium to high; platform-specific services.	High; depends on data export and standards.

6.4 Real provider examples

- **IaaS:** Amazon EC2 (AWS) or Microsoft Azure Virtual Machines.
- **PaaS:** Google App Engine, Azure App Service or Heroku.
- **SaaS:** Google Workspace, Microsoft 365 or Salesforce.

TASK 7 – Examine XaaS

7.1 XaaS and its variants

XaaS (Anything as a Service) is the general model in which any IT capability, not just infrastructure, platform or software, is consumed as a metered service over the internet. It

extends the IaaS/PaaS/SaaS idea to specialised functions so that the university can rent a capability instead of building and operating it.

Table 19: Common XaaS variants

Variant	What it provides	Example	Possible use at the university
DaaS (Desktop as a Service)	Virtual desktops delivered to any device.	Amazon WorkSpaces, Azure Virtual Desktop	Computer-lab desktops accessible from home or low-cost devices.
MBaaS (Mobile Backend as a Service)	Ready-made backend for apps: authentication, push notification, data sync.	Firebase	Backend for the new student mobile app.
FaaS (Function as a Service)	Run small event-driven functions without managing servers; billed per execution.	AWS Lambda, Azure Functions	Generate fee receipts, send result notifications.
DBaaS (Database as a Service)	Managed database with automatic backup, patching and replication.	Amazon RDS, Azure SQL Database	Database tier of the SIS and exam portal.
Security as a Service	Cloud-delivered security such as web firewall, identity protection and monitoring.	Cloudflare, Microsoft Defender	Protects public web and API endpoints.
DRaaS (Disaster Recovery as a Service)	Replication and failover of servers and data to a provider's recovery site.	Azure Site Recovery, AWS Elastic Disaster Recovery	Recovery site that the university does not currently have.

7.2 Two XaaS offerings selected for the university

- **DBaaS.** The SIS and exam portal depend on a reliable database, and running it in-house means patching, backups, replication and tuning by scarce staff. A managed multi-zone database gives automatic failover, scheduled backups, point-in-time recovery, encryption and easy scaling of read capacity for result-publication peaks. This improves availability, scalability and manageability while reducing administration cost.
- **DRaaS.** The university has no disaster-recovery site, and building a second data centre would be very expensive. DRaaS replicates data and server images to a second region and charges mainly for storage until a failover is needed. This provides measurable recovery targets (for example RPO of 15 minutes and RTO of 4 hours) at a fraction of the cost of an owned recovery site.

Other variants are useful later: FaaS for receipt and notification jobs and MBaaS for the mobile app (see Task 11), but DBaaS and DRaaS address the most serious present gaps, namely database reliability and the absence of disaster recovery.

7.3 Risks of over-reliance on many 'as-a-Service' providers

Table 20: Risks of depending on multiple providers and their mitigation

Risk	Explanation	Mitigation
Vendor lock-in	Proprietary services make it costly to move.	Use open standards (containers, PostgreSQL/MySQL, OpenAPI) and keep exportable backups.
Compounded availability	If several services are in series, overall availability is lower than any single SLA.	Calculate end-to-end availability; add redundancy and caching on critical paths.
Integration complexity	Many providers mean many interfaces, accounts and billing models.	Central identity (single sign-on), API gateway and infrastructure-as-code.
Security and compliance gaps	Responsibility is shared and differs per provider.	Review each provider's certifications, encrypt data, central logging and access reviews.
Cost sprawl	Small subscriptions add up; data-transfer fees are easily missed.	Budgets and alerts, tagging, regular cost reviews.
Provider failure or exit	A provider may change prices, terms or shut down.	Exit plan, contractual data-return clauses and periodic restore tests.

TASK 8 – Design the Cloud Solution Architecture

8.1 Architecture diagram

The architecture combines virtualization, web services and the service-model mix chosen in the earlier tasks. Users reach the system through an edge layer and a load balancer; the web/application tier, API layer and database tier run across three availability zones of a primary region, and data is replicated to a disaster-recovery region.

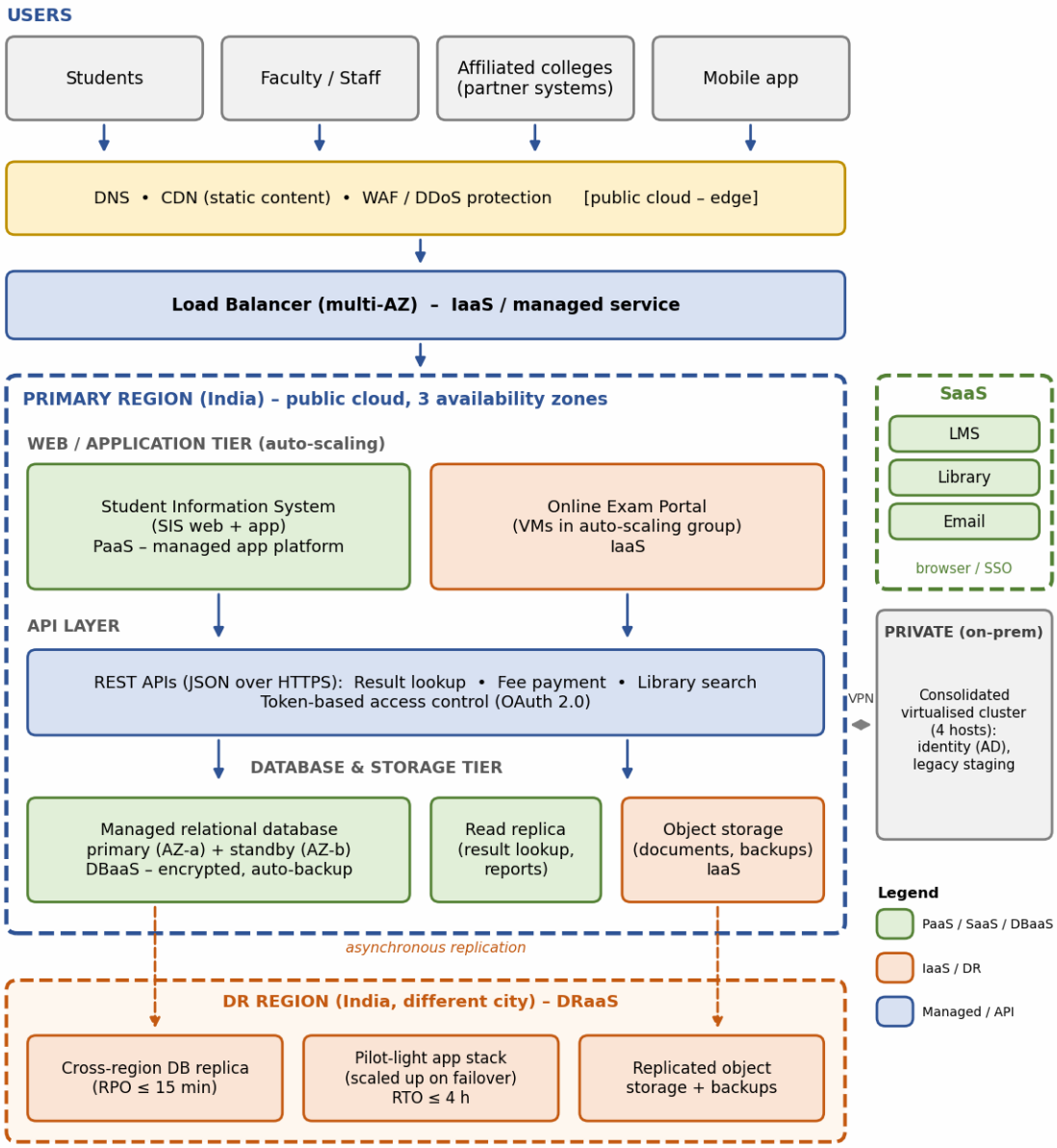


Figure 2: Proposed hybrid cloud architecture for the university (primary region, SaaS services, private cluster and DR region)

8.2 Deployment model and service model for each component

Table 21: Component-wise design decisions

Component	Deployment model	Service model	Justification
DNS, CDN and WAF	Public	Managed edge service	Caches static content, filters malicious traffic and absorbs DDoS attacks before they reach the applications.
Load balancer	Public	IaaS (managed)	Spreads traffic across zones and removes unhealthy instances automatically.
SIS web/application tier	Public	PaaS	Managed runtime with auto-scaling; no operating-system maintenance.

Exam Portal	Public	IaaS (VMs in auto-scaling group)	Full control for proctoring components and scheduled pre-scaling before exams.
API layer (result, fee, library)	Public	PaaS	Single, secured entry point for the mobile app and partner colleges; token-based access.
Database tier	Public (private subnet)	DBaaS	Multi-zone standby, automatic backups and encryption; read replica absorbs result-lookup load.
Object storage	Public	IaaS	Durable, inexpensive storage for documents and backups, replicated to the DR region.
LMS, Library, Email	Public	SaaS	Commodity functions run by specialists; no servers to maintain.
Identity and legacy cluster	Private (on-premise)	Virtualised cluster (4 hosts)	Keeps campus identity and legacy devices local; linked to the cloud by VPN.
DR region	Public (second city)	DRaaS	Cross-region replica and pilot-light stack for recovery (RPO ≤ 15 min, RTO ≤ 4 h).

8.3 Elasticity during admissions and exam peaks

- **Auto-scaling policies:** the web/application tier scales on CPU (target about 60%) and requests per instance, with a minimum of 2 instances per zone and an upper limit to control cost.
- **Scheduled scaling:** capacity is raised in advance of known events (admissions, result publication, exam timetable) so that users never wait for new instances to start.
- **Caching and CDN:** static pages and frequently requested results are served from the edge or an in-memory cache, reducing database load.
- **Read replica and connection pooling:** protect the database during result publication.
- **Queue or waiting-room control** at the exam portal to smooth sudden login surges.
- **Scale-in** after each peak, so the university pays for peak capacity only while it is used.

8.4 Security and availability measures

Table 22: Security and availability controls

Area	Measure
Access control	Role-based access control and least privilege; multi-factor authentication for administrators; single sign-on for users.
Encryption	TLS 1.2 or higher in transit; AES-256 encryption at rest with managed keys for databases, storage and backups.
Network security	Private subnets for application and database tiers; security groups; WAF and DDoS protection at the edge; VPN to the on-premise cluster.
Backup	Daily snapshots with point-in-time recovery for the database; backups copied to the second region; regular restore tests.
Multi-zone deployment	Application and database across three availability zones so that the loss of one zone does not stop service.

Disaster recovery	Cross-region replication and a pilot-light stack: RPO ≤ 15 minutes, RTO ≤ 4 hours.
Monitoring and audit	Central logs, metrics, alerts and audit trail; cost and security alerts.

TASK 9 – Hands-on Implementation and Documentation

The proof-of-concept implements the **result-lookup REST API** (one of the three university APIs from Task 5) as a Python Flask application, packages it as a **Docker container** (OS-level virtualization) and deploys the same container image to a **PaaS** service. The code below was written for this report. Replace the placeholders in the screenshot sections with your own screenshots from your own run.

9.1 Environment

Table 23: Environment used (update with your own details)

Item	Details
Local operating system	Windows / Linux / macOS (state yours)
Virtualization tool	Docker Desktop (state the version you installed)
Language and framework	Python 3.12, Flask 3.x, Gunicorn
Test client	curl and a web browser (Postman can also be used)
PaaS target	Google Cloud Run (free tier; billing account required) or an equivalent such as Azure App Service or Render

9.2 Source code and configuration

app.py

```
import os
from flask import Flask, jsonify

app = Flask(__name__)

# Sample in-memory data (a real system would query the database tier)
RESULTS = {
    "192370001": {
        "name": "Sample Student",
        "semester": 5,
        "gpa": 8.6,
        "subjects": [
            {"code": "CSA15", "title": "Cloud Computing and Big Data Analytics", "grade": "A"},
            {"code": "CSA11", "title": "Database Management Systems", "grade": "A+"},
        ],
    }
}

@app.route("/health")
def health():
    return jsonify(status="UP"), 200

@app.route("/api/v1/students/<reg_no>/results")
def get_results(reg_no):
    record = RESULTS.get(reg_no)
    if record is None:
        return jsonify(error="Student not found", registerNumber=reg_no), 404
```

```
    return jsonify(registerNumber=reg_no, **record), 200


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=int(os.environ.get("PORT", 8080)))
```

requirements.txt

```
flask==3.1.*
gunicorn==23.*
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
ENV PORT=8080
EXPOSE 8080
CMD ["sh", "-c", "gunicorn --bind 0.0.0.0:${PORT} app:app"]
```

9.3 Build and run the container (virtualization)

```
docker build -t university-results-api .
docker run -d --name results-api -p 8080:8080 university-results-api
docker ps
```

docker ps should list the container results-api with status “Up” and port mapping 8080.

9.4 Test the web service

Functional test – expected HTTP 200

```
curl -i http://localhost:8080/api/v1/students/192370001/results


HTTP/1.1 200 OK
Content-Type: application/json
{"gpa":8.6,"name":"Sample Student","registerNumber":"192370001","semester":5,
 "subjects":[{"code":"CSA15","grade":"A","title":"Cloud Computing and Big Data Analytics"},
            {"code":"CSA11","grade":"A+","title":"Database Management Systems"}]}
```

Failure case – expected HTTP 404

```
curl -i http://localhost:8080/api/v1/students/999999999/results


HTTP/1.1 404 NOT FOUND
Content-Type: application/json
{"error":"Student not found","registerNumber":"999999999"}
```

The output above is what the code returns (Flask lists JSON keys alphabetically). The headers in your own screenshot may show additional lines such as the server version and date.

9.5 Deploy the same container to a PaaS (Google Cloud Run example)

- Create or select a Google Cloud project, enable billing (the free tier still requires a billing account) and install the Google Cloud CLI.

- Sign in and set the project: gcloud auth login, then gcloud config set project <PROJECT_ID>.
- Deploy from the folder containing the Dockerfile; accept the prompts to enable the required services.

```
gcloud run deploy university-results-api --source . --region asia-south1 --allow-unauthenticated

# test the public URL printed at the end of the deployment
curl -i https://<your-service-url>/api/v1/students/1923700001/results
```

The service listens on the port given in the PORT environment variable, which the code already reads. The deployed service is automatically scaled by the platform, which is the PaaS behaviour discussed in Task 6.

9.6 Screenshots (to be inserted by the student)

Capture the following screenshots from your own machine and cloud account, and replace each placeholder. Annotate each one (for example, a red box around the status code or URL).

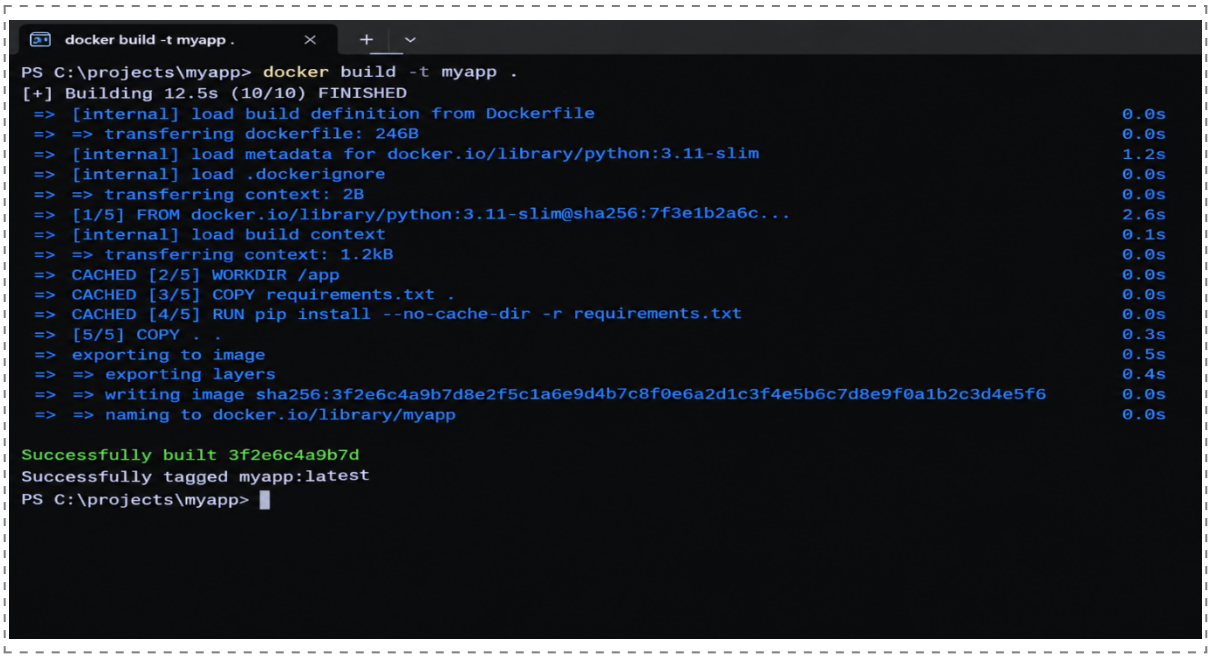


Figure 3: Screenshot – docker build output showing a successful build



Figure 4: Screenshot – docker ps showing the running container and port 8080



Figure 5: Screenshot – curl or browser output for the HTTP 200 result

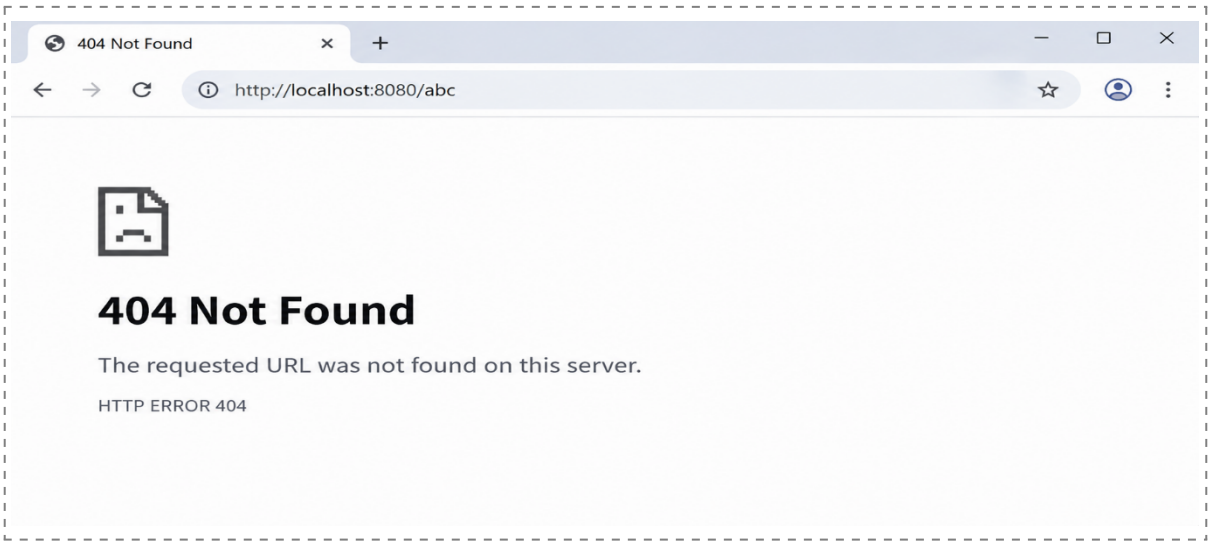


Figure 6: Screenshot – curl or browser output for the HTTP 404 failure case

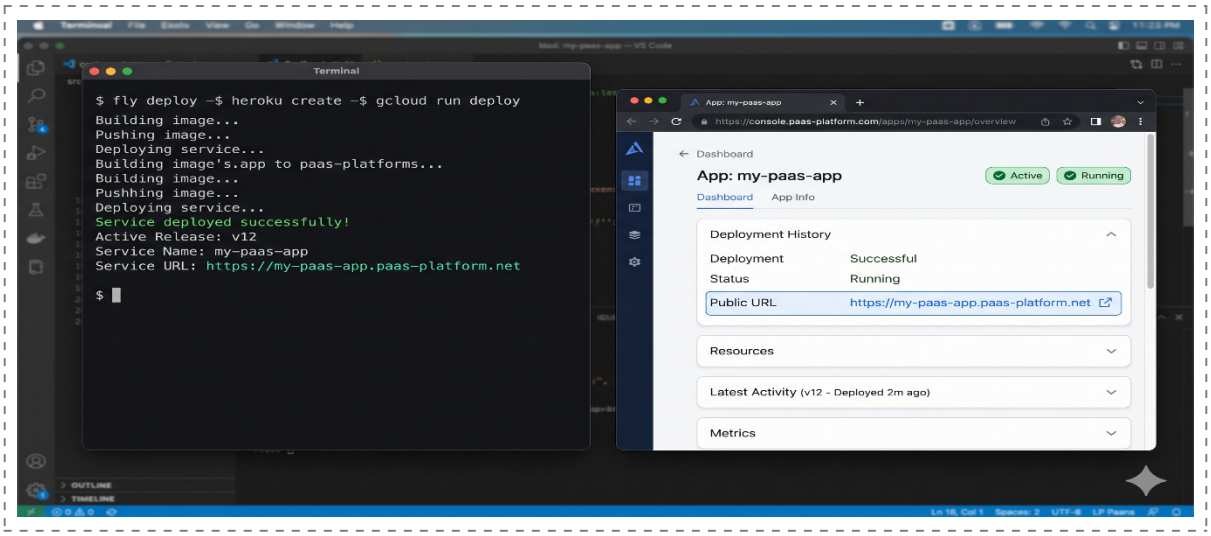


Figure 7: Screenshot – PaaS console or CLI output showing the deployed service and its URL

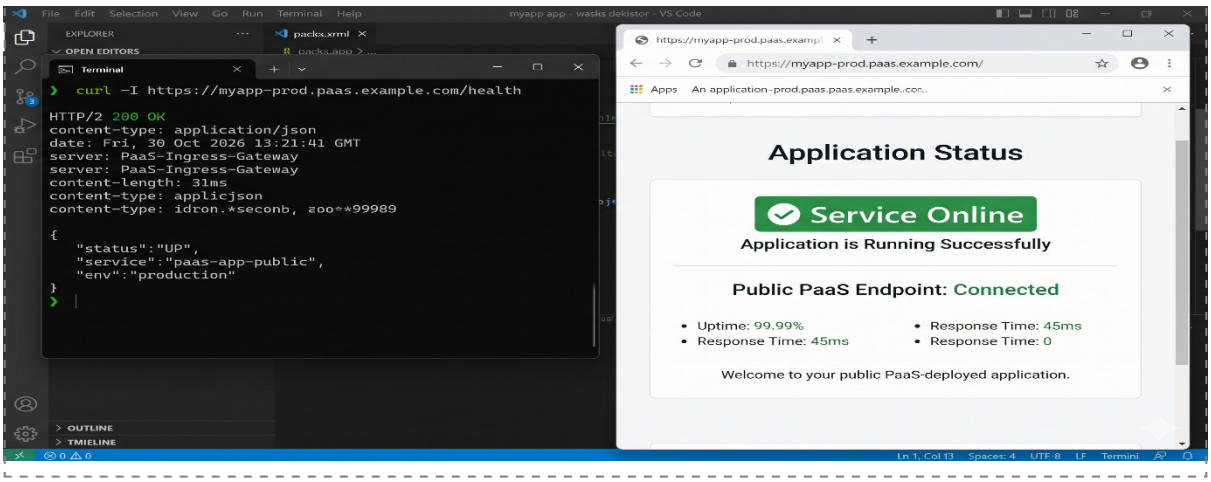


Figure 8: Screenshot – curl or browser test of the public PaaS URL

9.7 Issues and fixes

Typical problems and their fixes are listed below. Keep only the ones you actually faced, and add any others from your own run.

Table 24: Typical issues and resolutions (edit to match your experience)

Issue	Cause	Resolution
Browser cannot reach the container.	Application bound to 127.0.0.1 or port not published.	Run Flask on 0.0.0.0 and start the container with -p 8080:8080.
“Port is already allocated”.	Another process uses port 8080.	Stop that process or map another host port (for example -p 9090:8080).
Docker commands fail.	Docker daemon or Docker Desktop not running.	Start Docker Desktop and retry.
Cloud Run reports that the container failed to start.	Application did not listen on the PORT environment variable.	Read PORT in the code and bind to 0.0.0.0, as in app.py.
Deployment asks to enable APIs or billing.	Required cloud services not yet enabled.	Accept the prompt to enable the APIs and link a billing account.

TASK 10 – Evaluate and Recommend the Final Solution

10.1 Comparison of the three options

Table 25: On-premise, IaaS-based and PaaS/SaaS-based options compared

Criterion	A. On-premise (virtualised)	B. IaaS-based	C. PaaS/SaaS-based
Cost model	High CapEx; fixed running cost; sized for peak.	No CapEx; OpEx for VMs, partly reserved.	No CapEx; OpEx by usage and subscription.
Scalability	Limited to purchased hardware; 6–8 weeks to add.	Good; auto-scaling groups that the team must design.	Very good; largely automatic.
Availability	Single site; DR site would need extra capital.	Multi-zone and multi-region possible; team configures.	Built-in multi-zone; provider SLAs.
Security	Full control but all responsibility with the university.	Shared responsibility; OS and application security with the university.	Provider secures platform; university secures data and access.
Management effort	Highest (hardware, OS, applications).	High (OS and applications).	Lowest (applications and data only).
Time to deploy	Weeks to months.	Days to weeks.	Hours to days.

10.2 Three-year cost comparison

Assumptions

- All figures are **estimates in ₹ lakh** (1 lakh = ₹100,000) at constant prices, with no inflation or discounting; they must be validated with provider calculators and quotations.
- Load is the current one (about 27,000 users); each option includes disaster recovery for a fair comparison.
- **Option A** is sized for peak (about 10 hosts, from the peak calculation in Task 4), with a second small site for DR.
- **Option B** moves the applications to cloud virtual machines: reserved capacity for the baseline plus on-demand capacity for peaks.
- **Option C** uses PaaS and DBaaS for SIS and the exam workload and SaaS for LMS, Library and Email.
- Staff cost counts only administrators dedicated to infrastructure: 3, 2 and 1.5 full-time staff at ₹6 lakh each for A, B and C.

Table 26: Three-year cost estimate (₹ lakh)

Cost item	A. On-premise	B. IaaS	C. PaaS/SaaS
One-time: servers, storage, network (10 servers ₹60 + storage/network ₹15)	75	0	0

One-time: virtualisation and OS licences	12	0	0
One-time: DR site set-up	30	0	0
One-time: migration and refactoring	0	10	18
Total one-time (CapEx)	**117**	**10**	**18**
Yearly: compute, platform, DBaaS (B: reserved ₹24 + burst ₹6)	0	30	18
Yearly: storage, backup, network and DR (cloud)	0	18	7
Yearly: SaaS subscriptions (LMS, Library, Email)	0	0	14
Yearly: power, cooling, space and DR site running cost	16	0	0
Yearly: annual maintenance and support	8	0	0
Yearly: administrators	18	12	9
Total yearly (OpEx)	**42**	**60**	**48**
Three years of OpEx (3 × yearly)	126	180	144
Three-year total cost	**243**	**190**	**162**

Interpretation. Option C is the cheapest over three years: ₹81 lakh (33%) lower than on-premise and ₹28 lakh (about 15%) lower than IaaS. Option A looks attractive per year only if peak capacity is ignored; once it is sized for peak and given a DR site, most of its cost is idle hardware. Option B removes the capital cost but keeps the administration burden.

10.3 Main risks and mitigation

Table 27: Risk analysis and mitigation steps

Risk	Mitigation
Vendor lock-in	Use containers and open databases (for example PostgreSQL), keep APIs standard (OpenAPI), take regular exportable backups and keep an exit plan.
Data privacy and compliance	Choose Indian regions, encrypt data at rest and in transit, apply role-based access, sign data-processing terms with providers and keep audit logs (DPDP Act, 2023).
Downtime or internet failure	Multi-zone deployment, DR region, redundant internet links on campuses, SLAs and tested failover.
Skills gap	Train staff and obtain provider certifications, use managed services to reduce operational work, engage a migration partner for the first phase.
Cost overrun	Budgets and alerts, tagging by campus, auto-scaling limits, reserved capacity for the baseline, monthly cost review.
Migration failure or data loss	Phased migration, pilot users, verified backups, rollback plan and parallel run before cut-over.

10.4 Final recommendation

Recommended solution: Option C, a PaaS/SaaS-first hybrid cloud, using IaaS only where control is essential (the Exam Portal). The recommendation follows from the earlier analysis:

- Task 1 and Task 4: the 15% utilisation and peaks above 95% show that elasticity and pooling matter more than owning hardware; 4 hosts cover normal load but not peaks.
- Task 6: commodity applications (LMS, Library, Email) are best consumed as SaaS; the custom SIS fits PaaS; only the Exam Portal needs IaaS control.
- Task 7: DBaaS and DRaaS close the two largest reliability gaps.
- Task 10 cost analysis: lowest three-year cost and lowest administration effort.

Choosing the provider. The provider will not be chosen by popularity. A short list will be scored on these criteria: availability of two regions in India (needed for DR and data residency), education pricing, published SLAs, security certifications, support for standard open technologies (containers, standard databases) to limit lock-in, and results of a small proof-of-concept like the one in Task 9.

Migration roadmap

Table 28: Phased migration roadmap

Phase	Timeline	Applications and activities	Why in this order
0 – Preparation	Weeks 1–4	Assessment, provider selection, security and network set-up (identity, VPN, cost controls), staff training.	Foundation for all later phases.
1 – Quick wins	Months 2–3	**Email** to SaaS, then **Library** to SaaS.	Low risk, few dependencies, immediate savings; builds staff confidence.
2 – Learning platform	Months 3–5	**LMS** to SaaS (before the next term, with data migration and SSO).	Large user base, standard product; avoids exam and admission weeks.
3 – Exam workload	Months 5–7	**Exam Portal** on auto-scaling IaaS with load tests.	The largest peak-load pain point; must be proven before the next exam season.
4 – Core system	Months 7–12	**SIS** to PaaS with DBaaS and APIs using a phased (strangler) approach; DRaaS enabled.	Most critical and tightly coupled, so moved last when skills and controls are mature.
5 – Decommission	After month 12	Retire old servers; keep the 4-host private cluster for identity and legacy devices.	Releases hardware cost only after the cloud is stable.

DAY-4 DESIGN CHALLENGE

TASK 11 – Re-evaluate and Adapt the Design

Decision: MODIFY. The original hybrid, PaaS/SaaS-first design is kept as the base, but the compute, API, data and disaster-recovery layers are changed to meet 10 times the load, 99.9% availability, multi-tenant access and a limited budget. The sections below explain why.

11.1 Impact Analysis

What has changed

Table 29: Original problem compared with the changed requirement

Aspect	Original problem	Changed requirement
Users	About 27,000 internal students and staff on 4 campuses.	Same users plus the mobile app and 40 affiliated colleges (external organisations).
Load	Current peak (assumed about 1,000 requests/s and 5,000 concurrent users).	10 times the current peak: about 10,000 requests/s and 50,000 concurrent users.
Availability	Best-effort; no DR site.	99.9% (about 8.76 hours of downtime per year, or 43.8 minutes per month).
Access pattern	Internal, mostly campus-based.	Public internet, multi-tenant, programmatic (API) access.
Budget	Replace capital expenditure with operating expenditure.	Limited operating budget, so cost per unit of load must fall.

Original assumptions that are affected

- **Load:** the plan was sized for today's peaks, not 10 times more. Static sizing would make the cost explode.
- **Users:** external colleges and the mobile app bring unpredictable, continuous traffic rather than seasonal campus traffic.
- **Availability:** the original DR design (pilot light, RTO about 4 hours) tolerated a long recovery; 99.9% leaves little room.
- **Cost:** the earlier estimate (₹48 lakh per year at the current load) no longer applies.

Why web services and APIs now become critical

Earlier, APIs were an extra; now they are the product. The mobile app and each college's own systems will call the student-record and fee-payment services directly, so the APIs must be secure, versioned, rate-limited, monitored and highly available. The API layer becomes the single front door, and any weakness there affects every college and every student.

Effect of multi-tenant access by 40 colleges

- **Isolation:** one college must never see another college's data; every request must carry a verified tenant identity and every data access must be filtered by it.
- **Security:** more entry points and more administrators increase the attack surface; strong authentication, authorisation per tenant, encryption and audit logs are needed.
- **Fairness:** a busy or faulty college must not slow down others (noisy neighbour at application level), so per-tenant rate limits and quotas are required.

- **Compliance:** the university must be able to show who accessed what and keep personal data protected (DPDP Act, 2023).

11.2 Evaluation of the Original Design

Table 30: Can the original design handle the changed conditions?

Question	Finding
Can the service models and virtualization plan handle 10× load?	SaaS applications (LMS, Library, Email) scale on the provider's side, so they are fine. The PaaS SIS scales only within plan limits and a monolithic application tends to scale vertically (bigger instances), which is costly. The IaaS Exam Portal can scale out, but VMs take minutes to start, which is slow for sudden bursts. A single primary database becomes the bottleneck. **Conclusion: not sufficient without change.**
Are auto-scaling and a single active region enough for 99.9%?	Assume component availabilities of 99.99% (load balancer), 99.95% (application tier across zones) and 99.95% (managed database across zones). In series: 0.9999 × 0.9995 × 0.9995 = 0.9989, i.e. 99.89%, or about 9.6 hours of downtime a year, which is more than the 8.76 hours allowed. A pilot-light DR with an RTO of 4 hours would use almost half the yearly budget in one regional incident. **Conclusion: not sufficient.**
Does cost remain acceptable?	If the same PaaS design is simply made 10 times larger and left running all year, the yearly cost rises from ₹48 lakh to about ₹149 lakh (see the cost table in 11.4). **Conclusion: not acceptable for a limited budget.**
Would another service model or XaaS give better results?	Yes. Containers on managed orchestration start in seconds and pack more densely than VMs; a managed API gateway adds authentication, throttling and versioning; FaaS handles bursty background jobs (receipts, notifications) and is billed per execution; MBaaS covers push notifications and mobile authentication; DRaaS with a warm standby shortens recovery.

11.3 Adaptation or Redesign

Decision: retain, modify or redesign?

Table 31: Comparison of the three possible decisions

Option	Verdict	Reason
Retain	Rejected	Fails the 99.9% target (99.89%), is too costly at 10× load and has no tenant isolation or API management.
Redesign completely	Rejected	Unnecessary cost and delay. The SaaS applications, DBaaS, security baseline, identity and network design are still valid.
Modify	**Selected**	Keeps what works and changes only the layers that fail: compute, API, data scaling, tenancy and DR.

Revised architecture

Figure 9 shows the revised design. An API gateway authenticates every call and applies per-college limits. Stateless containers on a managed orchestration platform run across three availability zones and scale in seconds. Serverless functions handle receipts, notifications and result-PDF generation. A cache and a message queue protect the database, which is isolated per tenant, and a second region runs a warm standby.

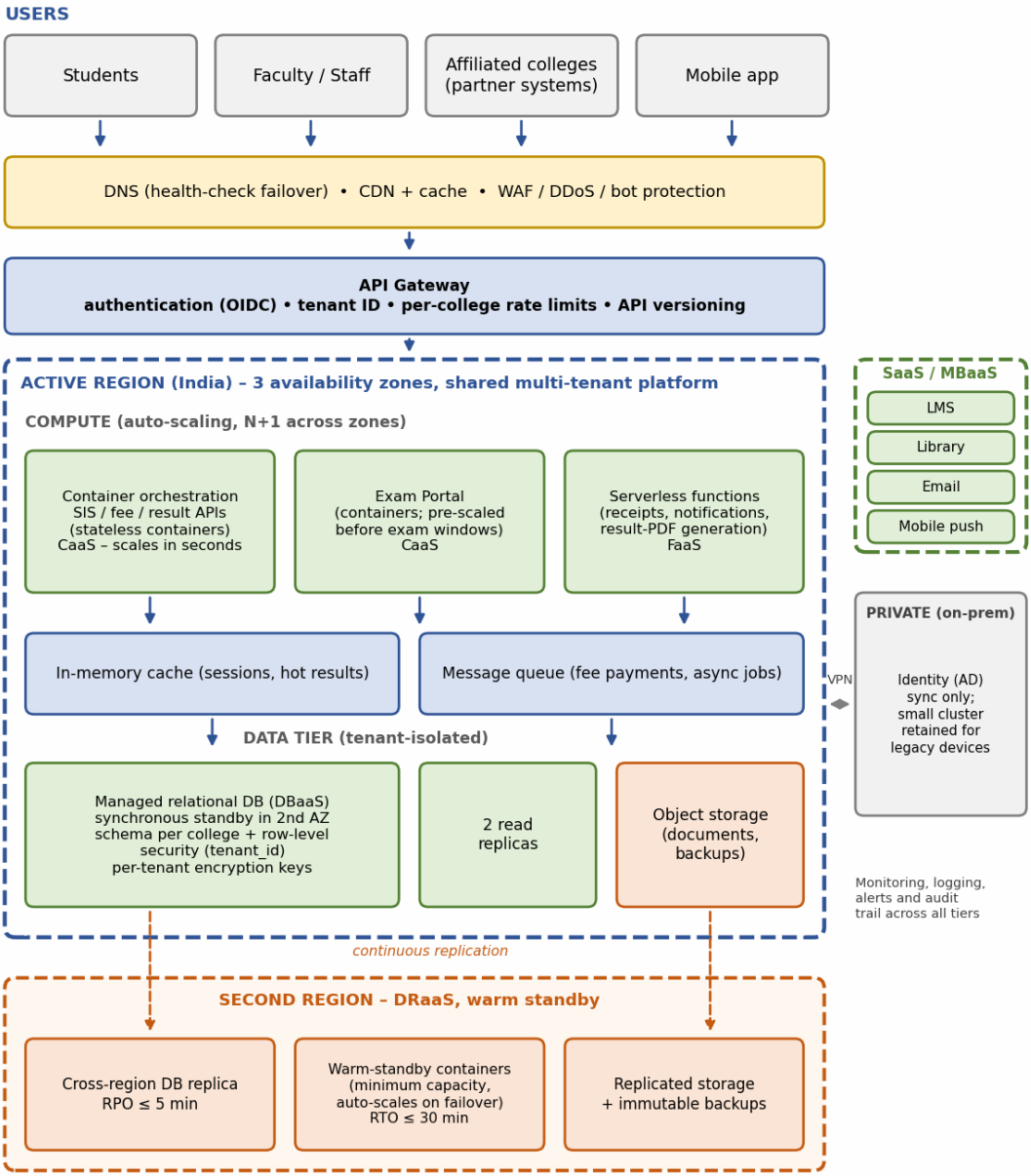


Figure 9: Revised (modified) cloud architecture for the mobile app and 40 affiliated colleges

What changed in the service-model mix and virtualization approach

Table 32: Changes between the original and revised design

Component	Original design	Revised design
Compute – SIS and APIs	PaaS application platform.	Stateless containers on managed orchestration (container-as-a-service), horizontally auto-scaled.
Compute – Exam Portal	IaaS virtual machines in an auto-scaling group.	Containers, pre-scaled on schedule before exams; VMs kept only for components needing OS agents.
API layer	REST endpoints on the application tier.	Managed API gateway with tenant authentication, rate limits and versioning.
Background jobs	Inside the application.	Serverless functions (FaaS), billed per execution.

Mobile support	None.	MBaaS-style services for push notification and authentication.
Database	DBaaS primary, standby and one read replica.	DBaaS with two read replicas, in-memory cache and message queue; schema per college with row-level security.
Disaster recovery	Pilot light, RTO about 4 hours, RPO 15 minutes.	DRaaS with warm standby, RTO about 30 minutes, RPO about 5 minutes.
LMS, Library, Email	SaaS.	Unchanged (SaaS).
Virtualization approach	Mostly VMs.	Mostly OS-level virtualization (containers); VMs for the database engine and legacy components.
On-premise cluster	4 hosts for identity and legacy staging.	Reduced role: identity synchronisation and legacy devices only.

11.4 Evidence-Based Defence

Capacity calculation

Table 33: Capacity calculation for 10× load (assumptions to be confirmed by load testing)

Item	Working
Current and new peak	Assumed current peak 1,000 requests/s; new peak = 10 × 1,000 = **10,000 requests/s**.
Capacity per container	Assume one container (2 vCPU, 4 GB) serves about 100 requests/s of typical JSON requests.
Instances for peak	10,000 ÷ 100 = 100; add 30% headroom: 100 × 1.3 = **130 instances**.
Surviving the loss of one of three zones	The remaining two zones must carry all 130 instances: 130 ÷ 2 = 65 per zone, so 65 × 3 = **195 instances** at peak.
Off-peak baseline	12 instances (4 per zone), scaled up before known events.
Database load	Assume 90% reads: 9,000 reads/s; with a 70% cache hit ratio 2,700 reads/s reach the database, about 900 per node across the primary and two replicas. Writes (about 1,000/s at peak) are buffered by the queue.

Compute cost with auto-scaling. Baseline: 12 × 8,760 h = 105,120 instance-hours. Peak events: assume 30 days a year at 195 instances: 30 × 24 × 195 = 140,400 instance-hours. Total = 245,520 instance-hours. At an assumed ₹3 per instance-hour this is about **₹7.4 lakh a year**. Keeping 195 instances running all year would be 195 × 8,760 = 1,708,200 instance-hours, about ₹51.2 lakh, so auto-scaling reduces compute cost by about **86%**.

Availability calculation

Table 34: Availability of the original and revised designs

Design	Working	Availability	Downtime per year
Allowed (target)	(1 − 0.999) × 8,760 h	99.9%	8.76 hours
Original	0.9999 × 0.9995 × 0.9995 = 0.9989; incidents last until repaired.	99.89%	about 9.6 hours (target missed)

Revised	Same components, but automatic health-check failover to a warm standby. Assume 4 incidents per year × RTO of 0.5 hour = 2 hours.	about 99.98%	about 2 hours (target met)

These are indicative values based on assumed component SLAs and incident counts. They must be confirmed with the chosen provider's published SLAs and a failover drill. Rolling deployments avoid planned downtime.

Cost comparison of the original and revised designs

Table 35: Yearly cost at 10× load (₹ lakh) – original design scaled up versus revised design

Cost item	Original design scaled to 10×	Revised design
Compute / platform (always on vs auto-scaled)	100	12
API gateway, WAF and CDN	3	8
Database (DBaaS)	16	16
Serverless and MBaaS	0	2
SaaS subscriptions (LMS, Library, Email)	14	14
Disaster recovery and backup	7	12
Monitoring and security tooling	0	4
Administrators	9	12
Total per year	**149**	**80**

- Revised design saves ₹69 lakh a year (46%) compared with simply scaling the original design.
- Three-year view: original scaled design = ₹18 lakh migration + 3 × 149 = **₹465 lakh**; revised design = ₹18 + ₹12 lakh extra refactoring + 3 × 80 = **₹270 lakh**, a saving of **₹195 lakh (about 42%)**.
- Cost per unit of load falls from ₹48 lakh per 1× (original at current load) to ₹80 lakh ÷ 10 = ₹8 lakh per 1×, a reduction of about **83%**.
- The extra yearly cost compared with the original at current load is ₹80 − ₹48 = ₹32 lakh. If recovered from the 40 colleges through usage-based charging (measured service), this is about ₹0.8 lakh (₹80,000) per college per year. This is an illustrative figure.

Security and multi-tenancy

- **Identity:** OAuth 2.0 / OpenID Connect; each token carries a tenant ID and role; the API gateway rejects calls without a valid tenant.
- **Data isolation:** shared database cluster with a separate schema per college plus row-level security on tenant_id. Very large or sensitive colleges can be given a dedicated database (silo model).

- **Encryption:** TLS in transit; encryption at rest with separate keys per tenant where required.
- **Fairness and abuse control:** per-college rate limits and quotas at the gateway, with WAF and bot protection at the edge.
- **Audit:** immutable, per-tenant access logs and alerts for unusual access.

Trade-offs

Table 36: Trade-offs of the revised design

Trade-off	Impact	How it is managed
Vendor lock-in	Managed gateway, DBaaS and serverless are provider-specific.	Containers and standard SQL for portability; API contracts in OpenAPI; limit proprietary services to non-core functions.
Higher complexity	More components: gateway, orchestration, cache, queue, functions.	Managed services, infrastructure-as-code, monitoring dashboards and staff training (counted in the extra administrator cost).
Migration effort	SIS and Exam Portal must be containerised and split into services.	Strangler approach: move one API at a time, run in parallel and keep the ₹12 lakh refactoring budget.
Extra cost of second region	Warm standby costs more than pilot light.	Keep standby at minimum capacity and scale on failover; part of the ₹80 lakh estimate.

Conclusion

The original design meets the original problem, but it cannot meet 10× load at 99.9% availability within a limited budget: it reaches only 99.89% availability, has no tenant isolation or API management, and would cost about ₹149 lakh a year. **Modifying** it, by adding an API gateway, container orchestration, serverless functions, caching and queueing, tenant isolation and a warm-standby DR region, reaches about 99.98% availability, supports 10,000 requests/s with loss of a zone, and costs about ₹80 lakh a year, which is 46% lower. A full redesign is unnecessary because the SaaS applications, DBaaS, security baseline and hybrid approach remain valid. The estimates rest on stated assumptions, so a load test and a failover drill should be carried out before go-live.